

Manual Tecnico

1. Arquitectura

```
app.py
-> vista.py
-> Lexer.py
-> sintatic.py
-> semantic.py
-> controlador.py
-> reportes.py
```

`app.py` es el punto de entrada y coordina el flujo. La vista Tkinter consume los resultados producidos por `Aplicacion`.

Evidencia de ejecucion

La ejecucion en consola muestra el resultado de las tres fases del analizador.

```

1 import argparse
2 from pathlib import Path
3
4 from lexer import AnalizadorLexico
5 from controlador import ControladorAnalisis
6 from semantic import AnalizadorSemantico
7 from sintaxis import ErrorSintactico, Parser
8
9
10 # This is the main application
11 class Analizador:
12     def __init__(self, ruta_archivo=None):
13         self.ruta_archivo = ruta_archivo
14         self.lexer = None
15         self.tokens = []
16         self.error_lexico = None
17         self.error_sintactico = None
18         self.error_semantico = None
19         self.controlador = None
20         self.datos_reportes = None
21
22     def ejecutar(self, ruta_archivo=None):
23         ruta = ruta_archivo or self.ruta_archivo or self.ruta_predeterminada()
24         self.ruta_archivo = ruta
25         texto = ruta.read_text(encoding="utf-8")
26
27         self.lexer = AnalizadorLexico(texto)
28         self.tokens = self.lexer.analizar()
29         self.error_lexico = self.lexer.error
30
31         if self.error_lexico:
32             return self.resultado()
33
34         try:
35             self.error_sintactico = self.controlador.analizar(self.tokens)
36         except ErrorSintactico as error:
37             self.error_sintactico = error
38             return self.resultado()
39
40         self.error_semantico = self.controlador.analizar(self.error_sintactico)
41         self.error_semantico = self.controlador.analizar(self.error_sintactico)
42         self.datos_reportes = self.controlador.datos_para_reportes()
43         return self.resultado()
44
45     def resultado(self):
46         return {
47             "tokens": self.tokens,
48             "error_lexico": self.error_lexico,
49             "error_sintactico": self.error_sintactico,
50             "error_semantico": self.error_semantico,
51             "datos_reportes": self.datos_reportes
52         }
53
54     def ruta_predeterminada(self):
55         return Path(__file__).parent / "datos" / "archivo.txt"
56
57
58 if __name__ == "__main__":
59     parser = argparse.ArgumentParser()
60     parser.add_argument("-a", "--archivo", type=str, help="Ruta del archivo a analizar")
61     args = parser.parse_args()
62     analizador = Analizador(ruta_archivo=args.archivo)
63     resultado = analizador.ejecutar(ruta_archivo=args.archivo)
64     print(resultado)

```

2. Analizador lexico

El archivo `Lexer.py` contiene:

- `Token`: representa un token con numero, lexema, tipo, linea y columna.
- `ErrorLexico`: representa un error estructurado.
- `AnalizadorLexico`: implementa el AFD manual.

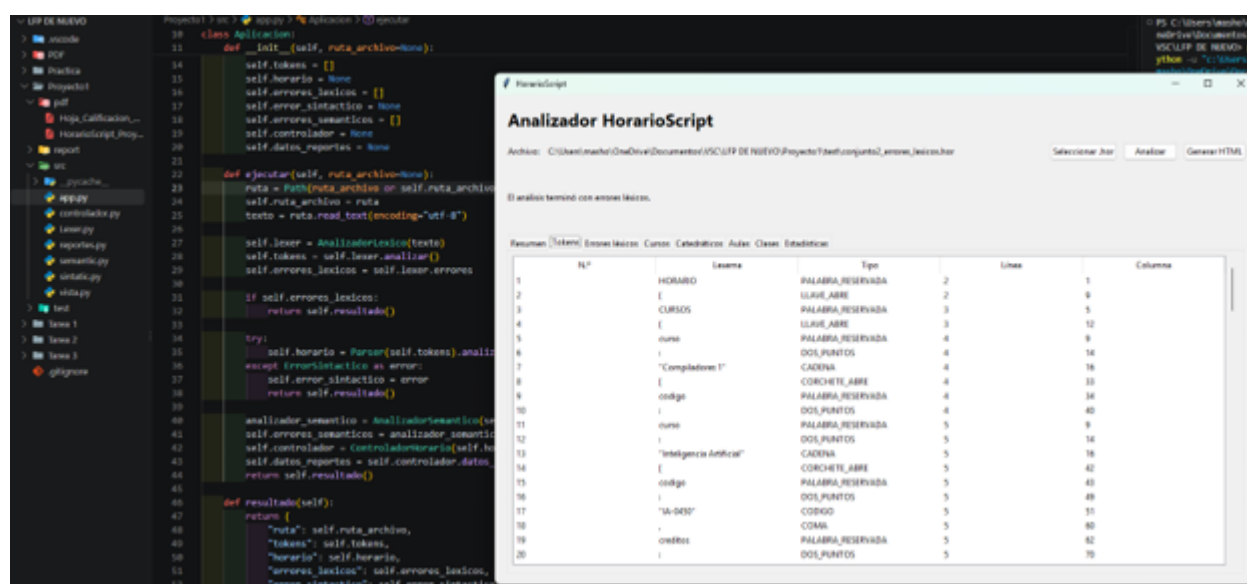
El metodo `siguiente_token()` observa el caracter actual, consume el lexema y devuelve un token. No se utiliza `re`, `split()` ni `find()` para la tokenizacion principal.

Tipos reconocidos:

- PALABRA_RESERVADA
- CODIGO
- CADENA
- HORA
- ENTERO
- DIA
- CATEGORIA
- delimitadores estructurales
- EOF

Los comentarios que comienzan con `##` se ignoran hasta el salto de linea o el final del archivo.

La tabla de tokens se presenta en la interfaz con la informacion de posicion producida por `Token`.



3. Analizador sintactico

`sintatic.py` contiene un parser descendente. Verifica la estructura:

```
HORARIO {
  CURSOS { ... };
  CATEDRATICOS { ... };
  AULAS { ... };
  CLASES { ... };
};
```

El parser consume los tokens con `esperar()`. Si el token actual no coincide con el esperado, genera `ErrorSintactico` con linea y columna.

4. Analizador semantico

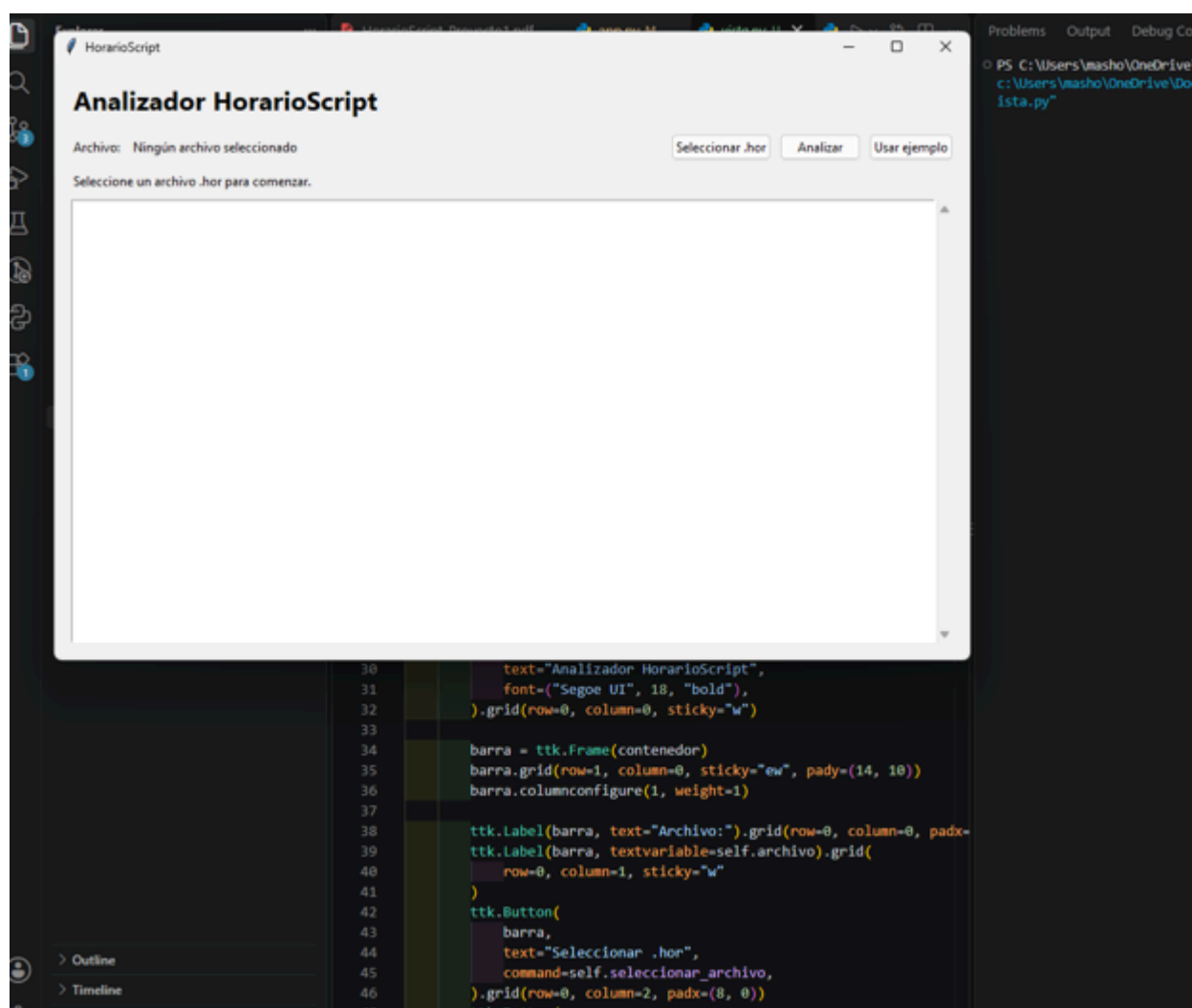
`semantic.py` construye tablas de simbolos para cursos, catedraticos y aulas. Luego verifica:

- declaraciones duplicadas;
- referencias a elementos no declarados;
- creditos y capacidades positivas;
- dias y categorias validas;
- hora inicial anterior a hora final;
- choques de catedratico;
- choques de aula.

5. Controlador

`controlador.py` convierte el horario en datos preparados para la vista y los reportes. Agrupa clases por dia, aula y catedratico, y calcula estadisticas generales.

La interfaz completa que consume estos datos se muestra a continuacion:



6. Ejecucion

```
python .\Proyecto1\src\app.py
python .\Proyecto1\src\app.py --consola
```

7. Limitaciones conocidas

La recuperacion sintactica continua despues del primer error aun puede ampliarse. Las funcionalidades opcionales como exportacion CSV/JSON, resaltado en tiempo real y multiples archivos no forman parte de la version actual.